

Omfattande Teknisk Dokumentation: Matematiska Modeller och Algoritmer för Batch-verifiering inom SIL

Denna dokumentation fungerar som ett tekniskt underlag och specifikation för programmatisk implementering av SIL-verifiering (Safety Integrity Level) i batch-processer. Modellen baseras på industristandarden IEC 61511 samt exidas etablerade metoder för att hantera diskontinuerliga processer i tillförlitlighetsblockdiagram och Markov-modeller.

1. Terminologi och Systemvariabler

För att bygga en beräkningsmotor krävs att följande parametrar definieras som datatyper (flyttal/integers) i programkoden:

Variabel (Kodnamn)	Datatyp	Mätenhet	Beskrivning
T_batch	Float	Timmar (h)	Tidslängden för en aktiv batch-cykel där processrisken är närvarande.
T_offline	Float	Timmar (h)	Tidsperioden mellan batcher då anläggningen är tömd/säker och risken är noll.
lambda_DU	Float	Fel per timme (1/h)	Farlig oupptäckt felfrekvens (Dangerous Undetected) för komponenten.
lambda_DD	Float	Fel per timme (1/h)	Farlig upptäckt felfrekvens (Dangerous Detected) via automatisk diagnostik.
C_batch	Float	Procent (0.0 - 1.0)	Testtäckning (Diagnostic Coverage) för det unika testet vid batch-slut.
T_proof	Float	Timmar (h)	Det ordinarie

Variabel (Kodnamn)	Datatyp	Mätenhet	Beskrivning
			manuella proof-test-intervallet (t.ex. 1-5 år).
C_proof	Float	Procent (0.0 - 1.0)	Testtäckning för det manuella proof-testet (ofta högre än C_batch, t.ex. 0.95).
T_mission	Float	Timmar (h)	Systemets totala livslängd (Mission Time, typiskt 10 till 20 år).

2. De Fyra Grundpelarna i Batch-modellering

2.1 Riskprofilen under Offline-perioden

Enligt IEC 61511 mäts risk som en funktion av frekvens och konsekvens. Om reaktorn eller tanken är tom under T_offline, kan ingen processhändelse (t.ex. en explosion eller ett överkok) inträffa på grund av kemikalier. Ur ett rent säkerhetsperspektiv är efterfrågansfrekvensen (Demand Rate) exakt noll. Därför definieras den *effektiva* sannolikheten för fel vid efterfrågan som noll under denna specifika tidsrymd:

```
Om System_Status == OFFLINE:
    Effektiv_PFD = 0
```

2.2 Ackumulering av utrustningsstress

Ett kritiskt misstag i enklare beräkningsmodeller är att frysa komponentens PFD under offline-tiden. Fysiska komponenter korroderar, utsätts för vibrationer, fukt och temperaturväxlingar även när processen vilar. Felfrekvensen lambda_DU är konstant över hela tidsaxeln. Det innebär att fel ackumuleras i bakgrunden, vilket leder till en diskontinuerlig "trappstegs-effekt" när en ny batch startar.

2.3 End-of-Batch Testning (Automatiska partial/full strokes)

När en batch avslutas stängs eller öppnas ofta ventilerna som en del av sekvensstyrningen. Detta är ett unikt tillfälle för styrsystemet att mäta svarstid, gränslägesbrytare och täthet. Detta fungerar i praktiken som ett högfrent proof-test, men med en begränsad testtäckning (C_batch) eftersom alla feltyper (t.ex. invändigt läckage) inte alltid kan detekteras vid en vanlig stängning.

2.4 Demand Mode-logik för Styrsystem (BPCS)

Programmet måste utvärdera om säkerhetsfunktionen är ett oberoende skyddslager (IPL). Om SIF-kretsen är separerad från processregleringen, genererar cykliska batch-startar *inte* en "High Demand". Varje batch-start är en planerad operation. En faktisk "Demand" (efterfrågan på säkerhetssystemet) uppstår endast om regleringen (BPCS) havererar under den aktiva batchen.

3. Matematiska Algoritmer (Steg-för-steg för kodning)

Nedan beskrivs den iterativa algoritmen för att beräkna den tidsbunden PFD-kurvan över systemets livslängd.

Steg 1: Beräkning av den momentana PFD-kurvan under en aktiv batch

För en given batch-cykel n som startar vid tidpunkten t_{start} , beräknas PFD vid tidpunkten t (där $t_{\text{start}} \leq t \leq t_{\text{start}} + T_{\text{batch}}$) enligt:

$$\text{PFD}(t) = \text{PFD_start}(n) + (\lambda_{\text{DU}} * (t - t_{\text{start}}))$$

Obs: Ovanstående är den linjära approximationen ($1 - e^{-\lambda t} \approx \lambda t$) som är industristandard för låga felfrekvenser.

Steg 2: Inkludering av offline-tid utan test

Om anläggningen går offline utan att ett test utförs, blir ingångsvärdet för nästa batch ($n+1$) högre eftersom utrustningen har åldrats under pausen:

$$\text{PFD_start}(n+1) = \text{PFD}(t_{\text{slut}}) + (\lambda_{\text{DU}} * T_{\text{offline}})$$

Steg 3: Reduktion vid End-of-Batch Test

Om styrsystemet utför ett godkänt batch-test vid slutet av batchen, reduceras den ackumulerade risken baserat på testtäckningen C_{batch} . Formeln för nästa batch-start blir då:

$$\text{PFD_after_test} = \text{PFD}(t_{\text{slut}}) * (1 - C_{\text{batch}})$$

$$\text{PFD_start}(n+1) = \text{PFD_after_test} + (\lambda_{\text{DU}} * T_{\text{offline}})$$

Steg 4: Integration för genomsnittligt PFD (PFD_{avg})

Det slutgiltiga SIL-värdet bestäms av PFD_{avg} . Eftersom PFD är matematiskt 0 under offline-perioderna (eftersom ingen demand kan uppstå), integrerar algoritmen endast över de aktiva batch-perioderna:

$$\text{Risk_Integral_Total} = 0$$

För varje cykel n från 1 till N :

$$\text{Risk_Integral_Total} += \text{Integralen av } [\text{PFD_start}(n) + \lambda_{\text{DU}} * t] \text{ från } 0 \text{ till } T_{\text{batch}}$$

$$\text{PFD_avg} = \text{Risk_Integral_Total} / T_{\text{mission}}$$

4. Pseudokod för Beräkningsmodul

Följande pseudokod kan direkt översättas till Python, C# eller C++ i ditt program:

```
def beräkna_batch_sil(T_batch, T_offline, lambda_DU, C_batch,
T_mission, T_proof, C_proof):
    t = 0
    tidssteg = 1.0 # 1 timme per iteration
    ackumulerad_risk_integral = 0.0
    nuvarande_pfd_start = 0.0

    medan t < T_mission:
        # 1. Aktiv Batch-fas
        aktiv_tid = 0
        medan aktiv_tid < T_batch och t < T_mission:
            momentan_pfd = nuvarande_pfd_start + (lambda_DU *
aktiv_tid)
            ackumulerad_risk_integral += momentan_pfd * tidssteg
            t += tidssteg
            aktiv_tid += tidssteg

            # Slutet på batchen - utför test
            pfd_vid_slut = nuvarande_pfd_start + (lambda_DU * T_batch)
            pfd_efter_batch_test = pfd_vid_slut * (1.0 - C_batch)

            # 2. Offline-fas (Risk-integralen ökar INTE eftersom
PFD_effektiv = 0)
            t += T_offline

            # Beräkna startvärdet för nästa batch efter offline-stressen
            nuvarande_pfd_start = pfd_efter_batch_test + (lambda_DU *
T_offline)

            # 3. Kolla om det är dags för ett stort manuellt Proof Test
            om t % T_proof == 0:
                nuvarande_pfd_start = nuvarande_pfd_start * (1.0 -
C_proof)

        pfd_avg = ackumulerad_risk_integral / T_mission
        risk_reduction_factor = 1.0 / pfd_avg

    return pfd_avg, risk_reduction_factor
```

5. Implementationsråd för Programvara

- **Numerisk precision:** Eftersom λ_{DU} ofta är i storleksordningen 10^{-6} eller

10^{-7} , bör variabler deklarerats som 64-bitars flyttal (Double i C#/C++, Float64 i Python) för att förhindra avrundningsfel vid långa integrationer över 20 år.

- **Arkitektur:** Separera beräkningskärnan (ovan algoritm) från databasen där komponentdata (från t.ex. exidas SERH-databaser) lagras.
- **Validering:** Verifiera din kod mot kända testfall. Ett system som körs 100% kontinuerligt utan batch-funktioner ska ge exakt samma resultat som standardformeln $PFD_{avg} = \frac{\lambda_{DU}}{T_{proof}^2}$ (vid 100% testtäckning).